Quantstamp

Venus - Flash Loans

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Money Market
Timeline	2025-10-15 through 2025-10-22
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	None
Source Code	• VenusProtocol/venus-protocol #5f264f6
Auditors	• Ibrahim Abouzied Auditing Engineer • Rabib Islam Senior Auditing Engineer • Paul Clemson Auditing Engineer

Documentation quality	High
Test quality	High
Total Findings	0
High severity findings	0
Medium severity findings	0
Low severity findings	0
Undetermined severity findings	0
Informational findings	0

Summary of Findings

The Venus Protocol is a money market on the Binance Smart Chain.

This audit focused on the new support for Flash Loans: a loan that requires no collateral, provided it is repaid within the same transaction. This upgrade is facilitated by changes to the `VToken` contract and the addition of a `FlashLoanFacet` to the Comptroller.

Flash loans are only permitted for whitelisted users on flashloan-enabled pools. The flash loan can include multiple assets. The user need not repay the loan in full, only the flash loan fees are required to be settled within the same transaction. If any unpaid tokens remain, they are taken on as a debt position with the same collateral requirements as traditional `VToken` borrows. Part of the flash loan fee is transferred to the `IProtocolShareReserve`, and the remainder is left in the `VToken` contract to act as the supplier's fee.

Our audit did not uncover any security vulnerabilities. Some of the attack vectors explored include circumventing access control and whitelisting, opening debt positions for another address without delegation, price manipulation, reentrancy, and the correctness of accounting during flash loans for fee calculation, token pricing, and debt position creation. While no exploits were discovered, we have provided recommendations to further improve the security and code quality.

Fix-Review Update: The Venus team has addressed S1 and S3, and acknowledged all other suggestions.

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

> **Disclaimer**
> Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence

- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
    2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

This audit is a diff audit of the Pull Request: [VEN-2895]: Flash loan implementation #545.

Commits: `dba9c600f563e22419b49717cd254e572ad5286c` through `5f264f6bad22db3e0a66dd228ea43cb77e55371b`.

Files Included

- `contracts/Comptroller/Diamond/facets/FacetBase.sol`
- `contracts/Comptroller/Diamond/facets/FlashLoanFacet.sol`
- `contracts/Comptroller/Diamond/facets/SetterFacet.sol`
- `contracts/Comptroller/Diamond/interfaces/IFlashLoanFacet.sol`
- `contracts/Comptroller/Diamond/interfaces/ISetterFacet.sol`
- `contracts/Comptroller/Diamond/Diamond.sol`
- `contracts/Comptroller/Diamond/DiamondConsolidated.sol`
- `contracts/Comptroller/ComptrollerInterface.sol`
- `contracts/Comptroller/ComptrollerStorage.sol`
- `contracts/external/IProtocolShareReserve.sol`
- `contracts/FlashLoan/interfaces/IFlashLoanReceiver.sol`
- `contracts/Tokens/VTokens/VToken.sol`
- `contracts/Tokens/VTokens/VTokenInterfaces.sol`
- `contracts/Utils/ErrorReporter.sol`
- `contracts/Utils/ReentrancyGuardTransient.sol`
- `contracts/Utils/TransientSlot.sol`

Operational Considerations

Sizeable flash loans are capable of causing significant fluctuations in the price of different assets on-chain. This can enable the exploitation of contracts reliant on certain price oracles that derive their data from on-chain sources.

The existing `approvedDelegates` mapping is reused to allow any approved delegate to also make flash loans on behalf of a user. It should be made clear to users that the updated version of the codebase will give delegates this power and give them time to remove delegations should they not wish to give existing delegates this power.

Flash loan fees are intended to be split between the protocol and existing liquidity providers within the protocol. However, it is possible that the protocol share of fees can be set to 100% of total fees, meaning liquidity providers earn nothing in fees from flash loans.

Key Actors And Their Capabilities

As it applies to the flash loan feature, privileged roles have access to the following functions:

- `SetterFacet.setWhiteListFlashLoanAccount()` : Manages the whitelist for flash loans.
- `VToken.setFlashLoanFeeMantissa()` : Configures the flash loan fee and the portion that goes to the protocol reserve.
- `VToken.setFlashLoanEnabled()` : Toggles the flash loan feature for the `VToken` .

Auditor Suggestions

S1 Lack of Emergency Pause for Flash Loan Mechanism

Fixed

Update

Marked as "Fixed" by the client.

Addressed in: `cc4b4f2313c4092d1deac08a31e2845404c4bc00` .

File(s) affected: `FlashLoanFacet.sol`

Description: In certain scenarios, it may be advantageous to temporarily pause the availability of the flash loan mechanism systemwide. While this is possible to do for individual pools, there is currently no easy method to do this across all VToken markets simultaneously.

Recommendation: Consider adding a pause mechanism that pauses all flash loan actions from the comptroller level.

S2 Missing Approved Amounts Validation

Acknowledged

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

```
Our flash loan feature now supports partial repayment. Users only need to approve the amount they wish to repay, not the full (borrowAmount + fee). Any unpaid portion will convert into a standard debt position. The only requirement is that the approved amount must at least cover the flash loan fee; otherwise, the transaction will revert. This check is already enforced in the _handleFlashLoan function.
```

File(s) affected: `FlashLoanFacet.sol`

Description: In `_executePhase2()` , the `receiver` 's `executeOperation()` function is called. This returns a `tokensApproved` array which is intended to represent the amount of the underlying that the `receiver` approves the `VToken` to spend in order to retrieve the underlying for each flashloan. However, this `tokensApproved` array is not validated, which may allow the VToken to over-spend or under-spend during recovery of the underlying for a given flashloan.

Recommendation: Check the allowances of the receiver towards the VTokens before `executeOperation()` is called and after. Calculate the difference and ensure that it corresponds to the `tokensApproved` values.

S3 Missing Input Validations

Fixed

Update

Marked as "Fixed" by the client.

Addressed in: `64c04cf0dc5fabf356e72b01ec54dc03873df14f` , `bb6db03bb7bddd4cbcd22c29669b9ee434156ca4` .

File(s) affected: `FlashLoanFacet.sol`

Description: For the `executeFlashLoan()` function:

- Check that the `onBehalf` parameter is not address zero.
- Consider setting a hard-coded maximum length of the `vTokens` array to ensure transactions can be completed without approaching block gas limits.

Recommendation: Consider adding the recommended input validation.

S4 Improve Use of Interfaces

Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

```
Having executeFlashLoan() in both the ComptrollerInterface and IFlashLoanFacet is by design. The ComptrollerInterface acts as the primary gateway, routing calls to the appropriate facet. The definition in IFlashLoanFacet simply declares the specific implementation. This is a standard pattern in our diamond architecture and does not cause any functional issues.
  To address the Certik audit finding VLW-05, which highlighted the problem of making core contracts abstract, we have mitigated the issue by removing the proposed VToken functions from the VTokenInterface. Their initial inclusion would have forced all inheriting contracts to be declared abstract.
  https://github.com/VenusProtocol/venus-protocol/pull/638/commits/49382e4057c0f52199d37557cfcf5ff0ac8efdf0
```

File(s) affected: `VTokenInterfaces.sol`, `ComptrollerInterface.sol`, `IFlashLoanFacet.sol`

Description:

- The `executeFlashLoan()` function is defined in both the `ComptrollerInterface` and the `IFlashLoanFacet`, making for two sources of truth.
- None of the new flash loan functionality is exposed by the interfaces in `VTokenInterfaces`.

Recommendation: Consider the following changes:

1. Keep the functions `executeFlashLoan()` and `authorizedFlashLoan()` in `IFlashLoanFacet` and have the `ComptrollerInterface` inherit from the interface.
2. Define the new `VToken` functions in `VTokenInterfaces`.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

- `778...8d0 ./ErrorReporter.sol`
- `321...b49 ./ReentrancyGuardTransient.sol`
- `3fb...d90 ./TransientSlot.sol`
- `9f4...bb0 ./VToken.sol`
- `177...7d4 ./VTokenInterfaces.sol`
- `cab...878 ./IFlashLoanReceiver.sol`
- `004...782 ./IProtocolShareReserve.sol`
- `192...ecd ./ComptrollerInterface.sol`
- `51d...1a2 ./ComptrollerStorage.sol`

- `e0f...273 ./Diamond/DiamondConsolidated.sol`
- `458...c22 ./Diamond/Diamond.sol`
- `cfd...15e ./Diamond/interfaces/ISetterFacet.sol`
- `d3b...f15 ./Diamond/interfaces/IFlashLoanFacet.sol`
- `79c...73d ./Diamond/facets/FlashLoanFacet.sol`
- `fa2...56f ./Diamond/facets/FacetBase.sol`
- `aa9...381 ./Diamond/facets/SetterFacet.sol`

Tests

- `36a...a93 ./flashLoan.ts`
- `f76...dff ./swapTest.ts`
- `063...0ce ./utils.ts`
- `146...3d1 ./EvilXToken.ts`
- `edb...f85 ./flashLoan.ts`
- `a14...3db ./scripts/deploy.ts`

Test Suite Results

Given the size of the `venus-protocol` test suite, only the flash loan tests were included for brevity. The full test suite runs successfully.

The test results were gathered by running `yarn test tests/hardhat/Comptroller/Diamond/flashLoan.ts`.

```
FlashLoan
    FlashLoan Multi-Assets
      ✔ Should revert if flashLoan is not enabled
      ✔ Should revert if the user is zero address
      ✔ Should revert if the market is not listed
      ✔ Should revert if contract is not whitelisted
      ✔ Should revert if array params are unequal
      ✔ should revert when requested flash loan amount is zero
      ✔ Should revert if receiver's executeOperation returns false (53ms)
      ✔ User has not supplied in venus - Should not create debt position if receiver repays full amount +
fee (65ms)
      ✔ User has not supplied in venus - should revert if repayment is insufficient (41ms)
      ✔ User has supplied in venus - Should not create debt position if repays full amount + fee (101ms)
      ✔ User has supplied in venus - Should create debt position if repays less than required (192ms)
      ✔ User has not enough supply in Venus and repays lesser amount (should revert) (69ms)
      ✔ Should revert with NotEnoughRepayment when repayment is less than total fee (71ms)
```

Code Coverage

The code coverage was gathered by running `npx hardhat coverage`. Only the in-scope files have been included.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
ComptrollerInterface.sol	100	100	100	100	
ComptrollerStorage.sol	100	100	100	100	
contracts/Comptroller/Diamond/	97.26	61.36	100	95.35	
Diamond.sol	97.26	61.36	100	95.35	109,228,229,230
DiamondConsolidated.sol	100	100	100	100	

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
FacetBase.sol	65.31	55.88	88.89	62.26	... 133,222,235
FlashLoanFacet.sol	100	80.77	100	94.12	62,90,296
SetterFacet.sol	87.76	77.08	85.71	87.93	... 636,662,710
VToken.sol	74.69	52.29	82.26	75.29	... 6,1877,1882
ReentrancyGuardTransient.sol	100	50	100	87.5	51
TransientSlot.sol	20	100	20	20	... 161,170,179
Address.sol	42.86	0	33.33	50	44,66,70,71

Changelog

- 2025-10-22 - Initial report
- 2025-10-28 - Final Report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over $200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.